

Transformations for Evolving Property Graph Schemas (Appendix)

Gauvain Devillez
University of Mons
Mons, Belgium
gauvain.devillez@umons.ac.be

Stefania Dumbrava
ENSIIE, INRIA, IRIF, SAMOVAR
Evry, France
stefania.dumbrava@ensiie.fr

Angela Bonifati
Lyon 1 University, CNRS Liris and IUF
Lyon, France
angela.bonifati@univ-lyon1.fr

PVLDB Reference Format:

Gauvain Devillez, Stefania Dumbrava, and Angela Bonifati.
Transformations for Evolving Property Graph Schemas
(Appendix). PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at
<https://github.com/GauvainD/GRAFT>.

APPENDIX

This appendix collects material deferred from the main text.

Appendix A illustrates the differences between relational and property graph schema evolution on the motivating example. Appendix B establishes GRAFT’s Jaccard similarity measure is exact (Theorem 3). Appendix C provides the schema-transformation algorithm and proves its termination, and Appendix D proves the optimality of greedy search under pruning for T -monotone meta-transformation sets. Appendices E-H report additional experiments: operation cost per edit (Appendix E), scalability with schema size (Appendix F), a real-world ICIJ case study (Appendix G), and a runtime breakdown across pipeline stages (Appendix H). Appendix I proves the completeness of unpruned GRAFT.

A RELATIONAL VS. PROPERTY GRAPH SCHEMA EVOLUTION

Figure 1 illustrates how the running example of Figure 1 might be structured in the relational model. Node types are converted to tables and edge types to foreign keys. In this setting, edit operations differ in their impact on the schema (see Table 1 for a comparison). The closest relational analogue to our meta-edit operations is the line of work on schema-modification operations (SMOs), in particular the PRISM workbench [2], where a relational schema evolves through a fixed catalogue of atomic operators and rewrite queries.

Note, however, that relational SMOs act on tables and attributes that can largely evolve independently, e.g., adding or dropping a column does not necessarily force changes elsewhere in the schema. In a property graph schema, structural and typing constraints are tightly coupled. Removing a node type leaves all incident edges

dangling, while renaming one rewrites the endpoint typing of every incident edge. Consequently, the evolution scenario of Figure 1 cannot be expressed as a set of independent edits: UpdTgt on hasModerator must precede the removal of the Moderator node type, and the relabelings interact with both. These ordering constraints are captured by Start* and Next*.

V1 (source)	V2 (target)
Forum(fid, title)	Forum(fid, title)
User(uid, firstName, lastName)	Person(uid, firstName, lastName)
Moderator(mid, firstName, lastName)	(dropped)
Place(pid, name)	Location(pid, name)
hasMember(fid→Forum, uid→User, creation)	hasMember(fid→Forum, uid→Person, creation)
hasModerator(fid→Forum, mid→Moderator)	hasModerator(fid→Forum, uid→Person)
isLocatedIn(uid→User, pid→Place)	isLocatedIn(uid→Person, pid→Location)

```
RENAME TABLE Place TO Location; RENAME TABLE User TO Person;
ALTER TABLE hasModerator DROP FOREIGN KEY (mid);
ALTER TABLE hasModerator RENAME COLUMN mid TO uid,
ADD FOREIGN KEY (uid) REFERENCES Person;
DROP TABLE Moderator;
```

Figure 1: Relational analogue of Figure 1.

Table 1: Relational SMOs vs. GRAFT operations.

	Relational	Property graph
Rename	RENAME TABLE: local; referencing FKs stay valid.	Relbl: endpoint types of incident edges change with it.
Re-point	Update the FK to a new referenced table.	UpdTgt must be applied before the old endpoint is removed.
Drop	DROP TABLE once no FK references it.	Rem leaves incident edges dangling.
Order	Explicit, detachable via declared FKs.	Implicit, mandatory (UpdTgt < Rem); enforced by Start*/Next*.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

B EXACTNESS OF JACCARD SIMILARITY

We restate here Theorem 3:

THEOREM 3. *On the extracted features, Jaccard similarity is exact for property graph schemas: for any two schemas S_1 and S_2 ,*

$$\text{sim}(S_1, S_2) = 1 \iff S_1 = S_2.$$

PROOF. A schema $S = (\mathcal{V}_S, \mathcal{E}_S, \rho_S)$ has disjoint finite sets $\mathcal{V}_S$ of vertex types and $\mathcal{E}_S$ of edge types, each type named uniquely within $\mathcal{V}_S \cup \mathcal{E}_S$, together with an endpoint map $\rho_S : \mathcal{E}_S \rightarrow \mathcal{V}_S \times \mathcal{V}_S$. We write $\rho_S(e) = (\xi(e), \tau(e))$, so that $\xi, \tau : \mathcal{E}_S \rightarrow \mathcal{V}_S$ return the source and target of each edge type; these are well defined since each edge type is fixed by its name. The proof additionally uses the property and label assignments $\pi : \mathcal{V}_S \cup \mathcal{E}_S \rightarrow \mathcal{P}(\text{Props})$ and $\lambda : \mathcal{V}_S \cup \mathcal{E}_S \rightarrow \mathcal{P}(\text{Labels})$ carried by the schema, whose names may recur across types. Two schemas are equal if and only if their vertex and edge sets, their endpoint maps, and their property and label assignments all coincide.

The extractor F keeps names in four separate namespaces, so a vertex name, an edge name, a property name, and a label name never collide even when they share the same string. It produces:

- **Names.** the vertex name of each $t \in \mathcal{V}_S$, the edge name of each $t \in \mathcal{E}_S$, every property name in $\bigcup_t \pi(t)$, and every label name in $\bigcup_t \lambda(t)$;
- **Incidences.** for each $t \in \mathcal{V}_S \cup \mathcal{E}_S$, the pairs (t, p) for $p \in \pi(t)$ and (t, ℓ) for $\ell \in \lambda(t)$; and for each $e \in \mathcal{E}_S$, the triples $(e, \xi(e), \text{src})$ and $(e, \tau(e), \text{tgt})$.

Structural features, such as two properties sharing a type, follow from the incidences; they do not affect exactness but refine the score when it is below 1.

Jaccard similarity equals 1 exactly when $F(S_1) = F(S_2)$, and equal schemas clearly yield equal feature sets. It remains to prove the converse, which we do by contrapositive: assuming $S_1 \neq S_2$, we exhibit a feature present in one set but not the other. As this is symmetric in the two schemas, we may fix the direction by swapping them if needed.

If $S_1 \neq S_2$, the components diverge in one of three ways.

- (1) *The vocabularies differ*, that is, $\mathcal{V}_{S_1} \neq \mathcal{V}_{S_2}$, $\mathcal{E}_{S_1} \neq \mathcal{E}_{S_2}$, $\bigcup_t \pi_1(t) \neq \bigcup_t \pi_2(t)$, or $\bigcup_t \lambda_1(t) \neq \bigcup_t \lambda_2(t)$. Then some name occurs in S_1 but not in S_2 , and its name feature, taken in the matching namespace, separates the two feature sets.
- (2) *The vocabularies agree but an attachment differs*. Suppose Case 1 does not hold and $\pi_1 \neq \pi_2$. Since $\mathcal{V}_{S_1} \cup \mathcal{E}_{S_1} = \mathcal{V}_{S_2} \cup \mathcal{E}_{S_2}$, some type t carries a property $p \in \pi_1(t) \setminus \pi_2(t)$. The pair (t, p) is a feature of S_1 but, as $p \notin \pi_2(t)$, not of S_2 . The case $\lambda_1 \neq \lambda_2$ is symmetric, via (t, ℓ) .
- (3) *The vocabularies and attachments agree but an endpoint differs*. Suppose Cases 1 and 2 do not hold and $\xi_1 \neq \xi_2$ (the case $\tau_1 \neq \tau_2$ is symmetric). Some $e \in \mathcal{E}_{S_1} = \mathcal{E}_{S_2}$ then has $\xi_1(e) \neq \xi_2(e)$. The triple $(e, \xi_1(e), \text{src})$ is a feature of S_1 ; since the only source feature of e in S_2 is $(e, \xi_2(e), \text{src})$, it is absent from S_2 .

The cases are exhaustive: once the vocabularies and attachments coincide, only the endpoint maps remain, which is Case 3. Hence $F(S_1) \neq F(S_2)$ whenever $S_1 \neq S_2$, which proves the contrapositive. $\square$

C GRAFT'S TRANSFORMATION GENERATION AND EXECUTION

This validation strategy is implemented in Algorithm 1 below.

Algorithm 1: Transform-Schema

Input: $E^*, \text{Start}^*, \text{Next}^*, S, T$
Output: $\mathcal{S}$: a set of schemas

```

1  $\mathcal{S} \leftarrow \emptyset$ 
2  $(\text{Start}, \text{Next}) \leftarrow \text{EVAL}(E^*, \text{Next}^*, \text{Start}^*, S, T)$ 
3  $\mathcal{M} \leftarrow \text{Derive} - \text{Transform}(\text{Start}, \text{Next})$ 
4 foreach  $M \in \mathcal{M}$  do
5    $S' \leftarrow S$ 
6   foreach  $e \in M$  do
7     if  $e$  is not valid with respect to  $S'$  then break
8      $S' \leftarrow \text{Apply}(e, S')$ 
9   end
10  if  $M$  is fully applied to  $S'$  then  $\mathcal{S} \leftarrow \mathcal{S} \cup \{(M, S')\}$ 
11 end
12 return  $\mathcal{S}$ 
```

Algorithm 1 grounds the meta-transformation to compute Start and Next (line 2) and derives the corresponding valid transformations (line 3). Each is then applied incrementally to the source schema (lines 6–9), with validity checked at every step (line 7). Only fully valid sequences produce output schemas (line 10).

THEOREM 1 (SCHEMA TRANSFORMATION TERMINATION). *For any finite input schemas, Algorithm 1 terminates and produces a finite set of transformation-schema pairs.*

PROOF. Since the input schemas are finite, the grounding of the meta-relations Start and Next produces finitely many edit operations. By Theorem 2, Algorithm 1 computes a finite set of transformation sequences. Algorithm 1 iterates over this finite set (line 4) and, for each sequence, applies finitely many edit operations (lines 6–9). Each loop therefore executes finitely many iterations. Hence, the algorithm terminates and produces a finite output. $\square$

D GRAFT'S OPTIMALITY UNDER PRUNING

Let $\mathcal{M}^*$ be a set of meta-transformations, S the source schema, and T the target schema. We write $\text{Reach}(S)$ for the set of schemas reachable from S using $\mathcal{M}^*$, and fix a similarity measure sim . Let T_b denote a most-similar reachable schema,

$$T_b \in \arg \max_{S_k \in \text{Reach}(S)} \text{sim}(S_k, T).$$

If sim is exact and $T \in \text{Reach}(S)$, then $T_b = T$.

To help with readability, we restate Definition 10 and Theorem 8.

DEFINITION 10 (T-MONOTONE META-TRANSFORMATION SET). *A set $\mathcal{M}^*$ of meta-transformations is T-monotone if, for every $S_k \in \text{Reach}(S)$ with $\text{sim}(S_k, T) < \text{sim}(T_b, T)$, some meta-transformation $M^* \in \mathcal{M}^*$ admits a grounding M with $\text{sim}(M(S_k), T) > \text{sim}(S_k, T)$.*

THEOREM 8 (GREEDY OPTIMALITY UNDER PRUNING). *Let $\mathcal{M}^*$ be T-monotone. Then for every $k \geq 1$, $\beta \geq 1$, $n_{\max} \geq 1$, and $\theta = 0$, Algorithm 2 with the greedy heuristic returns an optimal T_b .*

PROOF. Fix an execution of Algorithm 2. Let $h_t = \text{sim}(T', T)$ be the similarity of the best schema T' after t iterations, and let $h_b = \text{sim}(T_b, T)$. The best schema is replaced only by a strictly more similar one (line 14), so $h_1 \leq h_2 \leq \dots$ is non-decreasing. Every reachable schema has similarity at most h_b , hence $h_t \leq h_b$.

We first show that the current best schema T' always lies in the frontier S_{NEW} . When T' is updated (line 14), it was just generated by an expansion and inserted into S_{NEW} (line 16). It survives the top- k pruning of that expansion (lines 11–12), as it is the most similar schema in its generating set and $k \geq 1$. Schemas already in the frontier are never pruned. Thus T' can leave the frontier only by being selected for expansion (line 7). Suppose this happens while T' is non-optimal, i.e. $h_t < h_b$. By T -monotonicity, expanding T' produces a schema S_k with $\text{sim}(S_k, T) > h_t$. This schema becomes the new T' and re-enters S_{NEW} .

Assume $h_t < h_b$. Since T' attains the maximum similarity h_t over S_{NEW} , the greedy heuristic selects it (line 7, with $\beta \geq 1$). Expanding T' produces, by T -monotonicity, a schema S_k with $\text{sim}(S_k, T) > h_t$. This schema is new, as otherwise h_t would not be the best similarity so far. Therefore $h_{t+1} > h_t$. Each such improvement resets the no-improvement counter (line 14). As $n_{\max} \geq 1$, the loop does not halt while $h_t < h_b$.

By Lemma 1, $\text{Reach}(S)$ is finite, so sim takes finitely many values. As the loop only halts when $h_t = h_b$ and h_t strictly increases while $h_t < h_b$, we reach $h_t = h_b$ after finitely many iterations, and T' is optimal. If T is reachable, then $h_b = \text{sim}(T, T)$ and $\|T - T'\| = 0 \leq \theta$, so the returned transformation is exact. $\square$

E GRAFT’S EDIT OPERATION PERFORMANCE

Figure 2 reports the per-operation performance of the edit operations on the iBench synthetic dataset.

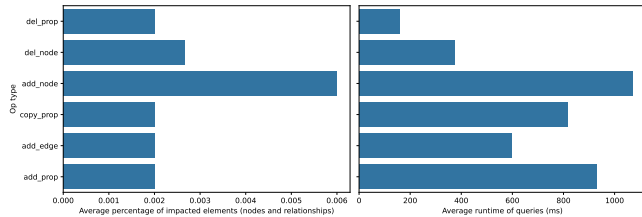


Figure 2: Average number of impacted elements per type of edit operation (left), average runtime (right).

On average, each edit operation affects only a small fraction of the graph (left plot). Node deletion has a slightly higher impact, since a deleted node may carry an incident edge that is removed along with it, affecting both a node and an edge. Node insertion has the largest impact, as it also adds the node’s label and an id property. We further distinguish copy_prop, which copies a property from one node to another, from add_prop, which adds a new property and so loads fresh data into the database.

The right plot shows the average runtime of each edit operation once translated into Cypher queries. The three deletion operations are the fastest, with node deletion slower than property deletion because it must also remove the node’s incident edges. The remaining operations are slower still, as each matches an element by id: a

property to its host node, an edge to its endpoints, or a new node to the node(s) it is split from or merged with.

These results show that the cost of an edit operation cannot be predicted statically, though coarse preferences can still be set. For example, the cost of deleting a node varies not only with the number of elements of its type, but also with the operations applied before it. Deleting the first endpoint of an edge costs more than deleting the second, since the first deletion already removes the shared edge. Estimating the a-priori cost of a transformation, or of a sequence of transformations, is thus a research question of its own. The framework can nonetheless support cost-aware search through a weighted strategy such as weighted_distance, which favors fewer transformations.

F GRAFT’S SCALABILITY

RQA. Which strategy offers the best trade-off between similarity to the target schema and runtime as the schema size grows?

To assess scalability, we augment both schemas with identical nodes and apply all strategies. Figure 3b reports normalized similarity improvement and Figure 3c, the corresponding runtimes.

For most datasets, both greedy and naive strategies achieve similarity scores that are optimal or near-optimal, where near-optimal denotes solutions whose similarity differs from the best observed value by less than a small margin. However, the runtime of naive increases sharply for Person-Data and Flight-Hotel, eventually resulting in timeouts. In contrast, the greedy heuristic consistently maintains substantially lower runtimes while preserving high-quality solutions as schema size increases.

The Flight-Hotel dataset exhibits less regular behavior: as the schema size increases, even the greedy strategy may fail to reach optimal similarity. This effect is due to the structure of the available meta-transformations, each of which applies exactly one edit operation. As a result, successive candidate schemas differ by a single edit, yielding a fine-grained search space in which many intermediate schemas offer small similarity improvements while preventing access to larger, non-monotonic gains.

The greedy strategy may repeatedly select locally improving schemas that do not lie on a globally optimal transformation path, thereby becoming trapped in local optima. This phenomenon explains its reduced effectiveness on the Flight-Hotel dataset.

Figure 3a plots normalized similarity improvement against θ . As expected, higher tolerance trades runtime for quality. Plateaus arise when θ exceeds the gap to the lowest similarity schema S_k with $\|T - S_k\| < \theta$. For db1p_to_amalgam1, the source schema is already 63% similar to the target, so the algorithm stops if $\theta = 0.5$.

Takeaway #1: greedy has the best similarity–runtime trade-off, matching naive without its timeouts, though highly granular meta-transformations (e.g., Flight-Hotel) may cause local optima.

G TRANSFORMING THE ICIJ DATABASE

We supplement the study of RQ5 with the real-world ICIJ Paradise Papers dataset which shows a smaller schema (5 nodes and 5 edges) and smaller database (2.02M nodes) but more extreme repartition between types (more than 78% of nodes are shared between two types). The transformations are derived from the rules in [3]. For

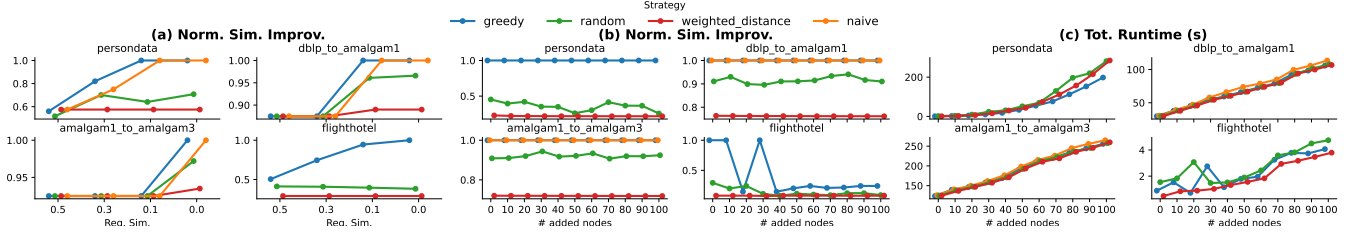


Figure 3: Y-axis: normalized similarity improvement (left and middle) and runtime; X-axis: values of θ (left), number of inserted nodes (middle and right).

Table 2: ICIJ meta-transformations.

ID	Meta-transformation
1	$\text{AddE}(e, n_1, n_2) \circ \text{AddProp}(n_2, p_2) \circ \text{Add}(n_2) \circ \text{AddProp}(n_1, p_1) \circ \text{Add}(n_1)$
2	$\text{AddProp}(n, p) \circ \text{Add}(n)$
6	$\text{AddE}(e_2, n_2, n_3) \circ \text{AddE}(e_1, n_1, n_2) \circ \text{Add}(n_3) \circ \text{AddProp}(n_2, p_1) \circ \text{Add}(n_2) \circ \text{Add}(n_1)$
9	$\text{AddProp}(n_3, p_2) \circ \text{AddE}(e_2, n_2, n_3) \circ \text{AddE}(e_1, n_1, n_2) \circ \text{Add}(n_3) \circ \text{AddProp}(n_2, p_1) \circ \text{Add}(n_2) \circ \text{Add}(n_1)$
10	$\text{AddProp}(n_1, p_k) \circ \dots \circ \text{AddProp}(n_1, p_1) \circ \text{Add}(n_1)$
14	$\text{AddE}(e, n, n) \circ \text{Add}(n)$
15	$\text{AddE}(e, n_1, n_2)$

the sake of readability, we group transformations by rule and sum similarity improvement and measurements.

Table 2 summarizes the ICIJ meta-transformations. Without idempotency, rules 1, 2, 14 and 15 are used for a resulting similarity of 73% to the target. With idempotency, GRAFT uses rules 1, 6, 9, 10 and 14 which are more complex and powerful, leading to 100% similarity.

Figure 4 shows that the highest increase in similarity among transformations is obtained by R10 which is also the longest runtime. However, because of the disparities between number of nodes per type, it is R9 which is the second longest runtime while being the fourth highest similarity increase. Runtime is higher than the iBench large-scale schema but remains within two minutes.

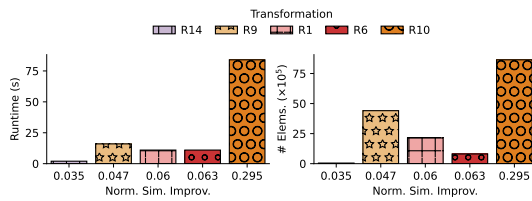


Figure 4: Evaluation of GRAFT's output on ICIJ.

H GRAFT'S RUNTIME BREAKDOWN

RQB. How is the runtime of GRAFT distributed across components?

We instrument each pipeline stage and report a runtime breakdown for each dataset using the greedy heuristic (pruning width 8, Jaccard similarity). This high-cost setting stresses all components. Times in Figure 5 are cumulative until termination.

We distinguish five steps: (i) *Edit operations generation* applies the rules to derive concrete edit operations and their Next/Start relationships from meta-transformations (line 2 in Algorithm 1); (ii) *Transformation generation* processes these tuples and contracts cliques to produce candidate transformations (lines 1–12 in Algorithm 1); (iii) *Schema generation* applies transformations to obtain new schemas (lines 4–12 in Algorithm 1); (iv) *Similarity computation* evaluates the similarity of each candidate schema to the target for pruning (line 14 in Algorithm 2); and (v) *Metagraph construction* inserts non-pruned schemas into the metagraph using signatures to avoid duplicates (lines 15–17 in Algorithm 2). This last step is dominated by database I/O.

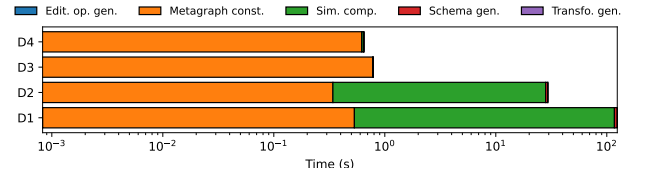


Figure 5: GRAFT runtime breakdown on Table 2 datasets.

As shown in Figure 5, similarity computation dominates the runtime, followed by schema generation and metagraph construction; edit operation and transformation generation are negligible. For smaller schemas, the IO overhead from the database overtakes the other timings which are exponentially smaller on those schemas. This is expected: meta-transformations generate a relatively small number of edit operations, whereas each new schema is obtained by applying a *sequence* of edits, leading to a much larger number of candidate schemas. Each such schema must be materialized and compared to the target, making similarity the main bottleneck. Pruning drastically reduces the number of schemas that reach the meta-graph, keeping its construction overhead manageable. Deduplication is applied to further reduce its size. Figure 6b shows the percentage of duplicated generated schemas that were filtered out.

Takeaway #2: GRAFT's runtime is dominated by schema materialization and similarity computation; edit generation is negligible.

I COMPLETENESS OF UNPRUNED GRAFT

THEOREM 2 (COMPLETENESS OF GRAFT WITHOUT PRUNING). Let S and T be schemas and $\mathcal{M}^*$ a finite set of meta-transformations whose variables range over elements of $S \cup T$. Without pruning, i.e., $k = \infty$ and $\theta = 0$, Algorithm 2 is complete with respect to target

schema reachability (Definition 9): if T is reachable from S using $\mathcal{M}^$, then Algorithm 2 returns a valid transformation from S to T .*

PROOF. In GRAFT, each schema reachable from the source is generated at most once and it stops when the threshold is reached. Hence, without pruning, if a path from S to T exists, T will eventually be generated during exploration. Then, the best approximation maintained by the algorithm satisfies $T' = T$, and therefore $\|T - T'\| = 0 \leq \theta$ (with $\theta = 0$). The loop stops and the algorithm terminates, returning a valid transformation from S to T . $\square$

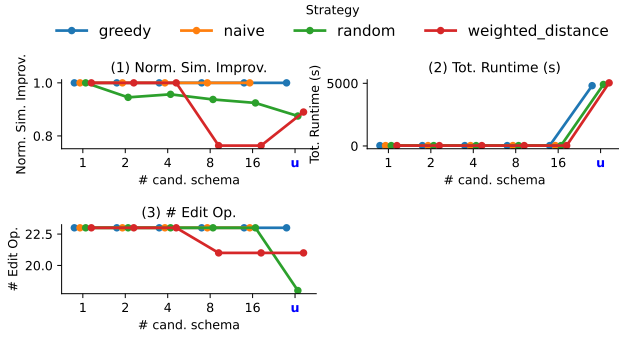


Figure 6: Performance on DBLP-Amalgam1 with pruning.

J PERFORMANCE OF GRAFT'S HEURISTICS WITH PRUNING ON DBLP-AMALGAM1

We compare the performance of the different heuristics with pruning on the DBLP-Amalgam1 dataset. Figure 6 reports: (a) the normalized similarity improvement, (b) the runtime, (c) and the number of edit operations for each heuristic. Similar to the results obtained for the Amalgam1-Amalgam3 dataset, greedy performs best.

K TECHNICAL DETAILS

This appendix complements Section 4. Section K.1 details meta-transformation generation and complexity. Section K.2 establishes the size bounds of the induced meta-graph. Section K.3 provides details concerning the formal properties of GRAFT.

K.1 Meta-Transformation Generation

Clique Detection. Algorithm 2 presents a basic version of the Bron-Kerbosch algorithm [1]. In our framework, maximal clique detection is used as a preprocessing step in Algorithm 1 to collapse permutation-equivalent edit operations into contracted edit operations, and as a postprocessing step to expand representative sequences when necessary. We construct a compatibility graph whose vertices are grounded edit operations and whose edges connect mutually compatible operations, i.e., operations e and e' such that each may follow the other and share the same predecessor context. Maximal cliques correspond to sets of edit operations that can be applied in arbitrary order.

Algorithm 2: Bron-Kerbosch

Input: P : set of grounded edit operations, R : current clique (initially $\emptyset$), X : excluded operations (initially $\emptyset$)
Output: Maximal cliques of mutually compatible edit operations

```

1 if  $P = \emptyset$  and  $X = \emptyset$  then
2   | output  $R$ 
3 end
4 foreach  $e \in P$  do
5   |  $P_e \leftarrow \{e' \in P \cap \text{Next}(e) \mid \text{pred}(e) = \text{pred}(e') \text{ and } e \in \text{Next}(e')\}$ 
6   | Bron-Kerbosch( $P_e, R \cup \{e\}, X \cap P_e$ )
7   |  $P \leftarrow P \setminus \{e\}$ 
8   |  $X \leftarrow X \cup \{e\}$ 
9 end
```

Worst-Case Complexity. Let D denote the length of the longest path in the edit-operation automaton, Δ the maximum number of successors of any edit operation, and k the maximum number of edit operations contracted into a single contracted edit operation.

THEOREM 3. Let D denote the maximum depth of the DAG of edit-operations, Δ the maximum number of successors of any edit operation, and k the maximum size of a contracted clique. Algorithm 2 runs in worst-case time $O\left(2^k \cdot \frac{(2^k \Delta)^{D-1}}{2^k \Delta - 1}\right)$.

PROOF. Assume first that no edit operations are contracted and that the successor relation forms a rooted directed acyclic graph with a single starting operation r . Partition edit operations into layers $E_0, \dots, E_D$ according to their distance from r .

In the worst case, every operation in E_i precedes every operation in E_{i+1} , with $|E_{i+1}| \leq \Delta$. An operation at depth i may then be explored via at most Δ^{i-1} distinct paths. The total number of explored states is therefore bounded by $1 + \Delta \frac{\Delta^D - 1}{\Delta - 1}$.

Consider now contraction. A contracted edit operation representing a clique of size k corresponds to 2^k subsets of underlying operations. These subsets can be enumerated in $O(2^k)$ time using incremental subset generation.

In the worst case, every operation belongs to a clique of size at most k , so each layer may contain up to $2^k \Delta$ effective successors. Substituting Δ with $2^k \Delta$ yields the stated bound. $\square$

THEOREM 1 (EXHAUSTIVE DUPLICATION-FREE META-TRANSFORMATION DERIVATION). Algorithm 1 enumerates the exact duplication-free transformation sequences that start with an edit operation in Start and respect the Next relation.

PROOF. Generation increases transformation length iteratively. Let $\mathcal{M}_t$ denote the set of generated transformations with $t \geq 1$ edit operations. The set $\mathcal{M}_1$ is obtained at line 2 and contains only transformations composed of one edit operation which is allowed by the Start rules by construction.

Assume $t > 1$ and consider the set $\mathcal{M}_t$. This set is generated after $t-1$ iterations of the loop at line 3. Let $M = \{e_t \circ e_{t-1} \circ \dots \circ e_0\} \in \mathcal{M}_t$.

M is obtained from a transformation M_{t-1} in $\mathcal{M}_{t-1}$ by instruction at line 8. By induction on t , $e_0 \in \text{Start}$. The loop at line 6 restricts e_t to be in $\text{Next}(e_{t-1})$, therefore ensuring Next is respected. Lastly,

line 7 ensures e_t is not an edit operation of M_{t-1} and thus that M does not contain duplicated edit operations.

For each t , if M_t is not empty, the loop at line 3 iterates and M_t is added to the result. If M_t is empty, then for each transformation $M_{t-1} = \{e_{t-1} \circ \dots \circ e_0\} \in \mathcal{M}_{t-1}$, either no edit operation is allowed to follow e_{t-1} by Next ($\text{Next}(e_{t-1}) = \emptyset$) or all candidate edit operations are already in M_{t-1} ($\forall e \in \text{Next}(e_{t-1}), e \in \{e_{t-1}, \dots, e_0\}$). By lines 6 and 7, the algorithm will then indeed produce an empty M_t . In this case, there is also no transformation of length greater than $t + 1$ and stopping computation from the condition in line 3 will not miss any transformation.

Therefore, all the transformations are produced and they all respect the stated constraints. $\square$

THEOREM 4 (WORST-CASE TIME COMPLEXITY OF META-TRANSFORMATION GENERATION). *Let n be the number of distinct edit operations. Then Algorithm 1 runs in $O(n!)$ in the worst case. With clique compression, this becomes $O(2^n)$.*

PROOF. Notice first that Algorithm 1 generates each transformation in $O(1)$. Its worst-case time complexity is thus equal to the worst-case number of transformations.

Let n denote the number of distinct edit operations. In the worst case, $|\text{Start}| = n$ and, for every edit operation e , $|\text{Next}(e)| = n$. Since duplicate edits are disallowed, a transformation of length t can be extended by at most $n - t$ operations.

Hence, the number of transformations of length t is bounded by the number of permutations of t elements drawn from n , i.e., $P(n, t) = \frac{n!}{(n-t)!}$. Summing over all lengths produces $\sum_{t=1}^n P(n, t) \in O(n! \cdot n)$. This worst case corresponds to a degenerate Next relation in which every edit operation may follow any other. When a subset of edit operations is pairwise compatible and commutative¹, we treat it as a *clique*. Such a clique can be compressed into a symbolic edit operation during enumeration. Since permutations of the same subset are equivalent under commutativity, it suffices to enumerate subsets rather than permutations, reducing the worst-case number of transformations to 2^n . $\square$

THEOREM 5 (WORST-CASE TIME COMPLEXITY OF TRANSFORMATION GENERATION). *Let n be the number of distinct grounded edit operations. Algorithm 1 runs in $O(2^n \cdot n)$ time in the worst case (with clique compression).*

PROOF. Let n be the number of distinct grounded edit operations produced at line 2. Since meta-edit variables range only over elements of the finite input schemas, n is finite and polynomial in the input size. By Theorem 4, with clique compression, Algorithm 1 (line 3) generates at most 2^n duplication-free transformations. Each transformation has length at most n . Algorithm 1 iterates once per transformation (line 4) and applies at most n edit operations per transformation (line 6), each in polynomial time.

Therefore, the overall worst-case running time is $O(2^n \cdot n)$, which is exponential in the number of grounded edit operations. $\square$

¹e.g., adding multiple properties to a node.

K.2 Meta-Graph Schema Search Space

THEOREM 6 (META-GRAPH SIZE BOUND). *Let n be the total number of distinct edit operations generated by the meta-transformations in $\mathcal{M}^*$. Then, the number of distinct schemas reachable under a finite set of meta-transformations is bound by 2^n . Consequently, the meta-graph contains at most 2^n vertices and at most $|\mathcal{M}^*| \cdot 2^n$ edges.*

PROOF. Let S be a schema in the meta-graph and let n be the total number of distinct edit operations generated by the meta-transformations. This schema is either one of the input schemas or obtained from one of them using a transformation. These transformations, by construction, cannot insert elements that are not present in one of the input schemas.

Let E be the set of all the n edit operations produced by the meta-transformations, and let S_i be a schema in the meta-graph. There is a sequence of unique edit operations that transform the source S into S_i . Indeed, consider the shortest path in the metagraph from S to S_i and let M be the composition of these transformations. M is a transformation too since it is a composition of edit operations. It contains at most n distinct edit operation and transforms S into S_i .

Thus, the total number of schemas in the metagraph is bounded by the total number of sequences of edit operation, which is $O(n!)$. However, every permutation of the same edit operations leads either to the same schema (the sets of objects added and removed are the same) or is an invalid sequence (e.g., setting a node property before adding it). Therefore, the number of schemas in the metagraph is bounded by the number of sets of edit operations, that is $O(2^n)$. A given transformation applied to a schema can only produce one schema. The meta-graph can thus have at most 2^n edges per meta-transformation, that is, at most $|\mathcal{M}^*| \cdot 2^n$. $\square$

K.3 GRAFT Theoretical Properties

THEOREM 4 (GRAFT TERMINATION). *Let S be a finite set of schemas and $\mathcal{M}^*$ a finite set of meta-transformations whose variables range over elements of $S \cup \{T\}$. Then Algorithm 2 terminates.*

PROOF. We prove termination by showing that only finitely many distinct schemas can be generated.

By Theorem 2, grounding the meta-transformations over the input schemas yields a finite set $\mathcal{M}$ of duplication-free transformations. By construction, transformations do not contain duplicate edit operations (Theorem 1), and each edit operation refers only to schema elements occurring in the input schemas. Hence, no transformation can introduce fresh symbols.

Let T be the finite set of schema elements (nodes, edges, labels, properties) occurring in the input schemas. Every schema generated during the execution of the algorithm is obtained by applying a finite sequence of grounded transformations to the source schema. Since transformations cannot introduce elements outside T , every generated schema is a subset of T . Therefore, by Theorem 6, the number of distinct reachable schemas is finite.

The algorithm maintains a set G containing all previously generated schemas. No schema is ever removed from G , and a schema is added to S_{NEW} only if it is not already contained in G . Hence, each schema can be inserted into S_{NEW} at most once.

Since the total number of distinct reachable schemas is finite, S_{NEW} can grow only finitely many times. Once all schemas that can

be reached have been added to G , no new schema can be generated, and S_{NEW} strictly decreases at each iteration. Therefore, the while-loop at line 8 terminates.

All remaining loops iterate over finite sets ($\mathcal{M}^*$ and $\mathcal{S}'$), and thus also terminate. Hence, the algorithm terminates. $\square$

THEOREM 7 (GRAFT TIME COMPLEXITY). *Let d denote the minimum number of transformation steps required to map S to T , b the branching factor induced by grounded meta-transformations, and C the cost of executing Algorithm 1. Without pruning, Algorithm 2 explores $O(b^d)$ schemas in the worst case, with runtime $O(b^d \cdot C)$. With pruning and width k , at most k^i schemas are retained at depth i , leading to the runtime bound $O(k^d \cdot b \cdot (C + \log k))$.*

PROOF. We model the execution of Algorithm 2 as a search tree of depth d , where each level corresponds to one transformation.

Without pruning, at each step, applying the grounded meta-transformations to a schema produces at most b successor schemas. In the worst case, the full search tree up to depth d is explored, generating $\sum_{i=1}^d b^i = O(b^d)$ distinct schemas. For each generated schema, Algorithm 1 is executed once, at cost C . Hence, the overall runtime is $O(b^d \cdot C)$.

When pruning is enabled, at depth i the algorithm retains at most k^i schemas. Expanding these schemas yields at most $k^i \cdot b$ candidates. For each candidate, Algorithm 1 is run at cost C , while keeping the top- k newly produced schemas by similarity to T . This selection can be implemented using a size-bounded heap, with $O(\log k)$ update cost per candidate. Summing over all depths gives a total cost of $\sum_{i=0}^{d-1} k^i \cdot b \cdot (C + \log k)$ and bound $O(k^d \cdot b \cdot (C + \log k))$. $\square$

REFERENCES

- [1] Coenraad Bron and Joep Kerbosch. 1973. Finding All Cliques of an Undirected Graph (Algorithm 457). *Commun. ACM* 16, 9 (1973), 575–576.
- [2] Carlo Curino, Hyun Jin Moon, and Carlo Zaniolo. 2008. Graceful database schema evolution: the PRISM workbench. *Proc. VLDB Endow.* 1, 1 (2008), 761–772.
- [3] ICIJ. 2022. The Offshore Leaks Database. <https://github.com/ICIJ/offshoreleaks-data-packages>.